

KRISH AI - SQL Knowledge Base

Plain selectable text knowledge base for Krish AI. Designed for PyMuPDF extraction and RAG chunking.

SQL Fundamentals

SQL means Structured Query Language. It is used to create, read, update, delete, and analyze data in relational databases.

Database: a collection of organized data. Table: data arranged in rows and columns. Row: one record. Column: one attribute. Primary key: a value or combination of values that uniquely identifies a row.

Common SQL systems include MySQL, PostgreSQL, SQL Server, Oracle Database, and SQLite.

SELECT and FROM

```
SELECT column1, column2 FROM employees;
```

```
SELECT * FROM employees;
```

SELECT chooses columns. FROM specifies the table. The asterisk means all columns.

Filtering with WHERE

```
SELECT * FROM employees WHERE salary > 50000;
```

```
SELECT * FROM employees WHERE department = 'Sales';
```

```
SELECT * FROM employees WHERE salary BETWEEN 30000 AND 60000;
```

```
SELECT * FROM employees WHERE department IN ('Sales', 'HR');
```

```
SELECT * FROM employees WHERE name LIKE 'A%';
```

WHERE filters rows. Common operators include =, <>, >, <, >=, <=, BETWEEN, IN, LIKE, IS NULL, and IS NOT NULL.

AND, OR, NOT

```
SELECT * FROM employees WHERE department = 'Sales' AND salary > 50000;
```

```
SELECT * FROM employees WHERE department = 'Sales' OR department = 'HR';
```

```
SELECT * FROM employees WHERE NOT department = 'HR';
```

AND requires all conditions. OR requires at least one condition. NOT reverses a condition.

ORDER BY and LIMIT

```
SELECT * FROM employees ORDER BY salary DESC;
```

```
SELECT * FROM employees ORDER BY department ASC, salary DESC;
```

```
SELECT * FROM employees ORDER BY salary DESC LIMIT 5;
```

ASC sorts ascending. DESC sorts descending. LIMIT restricts returned rows in systems that support it.

INSERT, UPDATE, DELETE

```
INSERT INTO employees (id, name, department, salary) VALUES (1, 'Arun', 'Sales', 45000);
```

```
UPDATE employees SET salary = 50000 WHERE id = 1;
```

```
DELETE FROM employees WHERE id = 1;
```

INSERT adds rows. UPDATE changes rows. DELETE removes rows. Always verify UPDATE and DELETE conditions before running them.

Aggregate Functions

```

SELECT COUNT(*) FROM employees;
SELECT SUM(salary) FROM employees;
SELECT AVG(salary) FROM employees;
SELECT MIN(salary), MAX(salary) FROM employees;

```

COUNT counts rows or values depending on the expression. SUM adds values. AVG calculates an average. MIN and MAX return the smallest and largest values.

GROUP BY and HAVING

```

SELECT department, COUNT(*) AS employee_count FROM employees GROUP BY department;

SELECT department, AVG(salary) AS average_salary FROM employees GROUP BY department HAVING
AVG(salary) > 50000;

```

GROUP BY creates groups for aggregation. HAVING filters groups after aggregation. WHERE filters rows before grouping.

NULL Values

NULL represents a missing, unknown, or unavailable value. NULL is not the same as zero or an empty string.

```

SELECT * FROM employees WHERE manager_id IS NULL;

SELECT * FROM employees WHERE manager_id IS NOT NULL;

```

Constraints and Tables

PRIMARY KEY uniquely identifies rows. FOREIGN KEY connects related tables. NOT NULL prevents missing values. UNIQUE prevents duplicate values. DEFAULT supplies a value when none is given. CHECK enforces a condition when supported.

```

CREATE TABLE employees (id INT PRIMARY KEY, name VARCHAR(100) NOT NULL, department
VARCHAR(50), salary DECIMAL(10,2), hire_date DATE);

```

Joins

INNER JOIN returns matching rows from both tables. LEFT JOIN returns every left-table row and matching right-table rows. RIGHT JOIN returns every right-table row where supported. FULL OUTER JOIN returns rows from both sides where supported.

```

SELECT e.name, d.department_name FROM employees e INNER JOIN departments d ON e.department_id
= d.department_id;

SELECT e.name, d.department_name FROM employees e LEFT JOIN departments d ON e.department_id
= d.department_id;

```

Subqueries

A subquery is a query inside another query. It can be used in WHERE, FROM, SELECT, or other clauses depending on the database.

```

SELECT name, salary FROM employees WHERE salary > (SELECT AVG(salary) FROM employees);

```

CASE Expression

```

SELECT name, salary, CASE WHEN salary >= 60000 THEN 'High' WHEN salary >= 40000 THEN 'Medium'
ELSE 'Low' END AS salary_level FROM employees;

```

CASE creates conditional output and is useful for categories and business rules.

Common Table Expressions

```

WITH high_salary AS (SELECT * FROM employees WHERE salary > 60000) SELECT * FROM high_salary;

```

A CTE creates a temporary named result and can make complex queries easier to read.

Window Functions

```
SELECT name, department, salary, ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS rank_in_department FROM employees;
```

Window functions calculate across related rows without collapsing them. Common functions include ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, SUM, and AVG.

Views and Indexes

```
CREATE VIEW sales_employees AS SELECT * FROM employees WHERE department = 'Sales';
```

A view is a saved query that behaves like a virtual table. An index can improve lookup performance but uses storage and can increase the cost of data modifications.

Transactions

```
START TRANSACTION; UPDATE accounts SET balance = balance - 100 WHERE id = 1; UPDATE accounts SET balance = balance + 100 WHERE id = 2; COMMIT;
```

Transactions group operations into one unit of work. COMMIT saves changes. ROLLBACK cancels uncommitted changes. Syntax varies by database.

SQL Query Processing Order

The logical processing order is generally FROM and JOIN, WHERE, GROUP BY, HAVING, SELECT, DISTINCT, ORDER BY, and LIMIT or its equivalent.

SQL for Data Analysts

Data analysts commonly use SQL to filter data, join tables, calculate metrics, find duplicates, summarize data, investigate trends, and prepare datasets for dashboards.

```
SELECT department, COUNT(*) AS employees, AVG(salary) AS avg_salary FROM employees GROUP BY department ORDER BY avg_salary DESC;
```

```
SELECT e.name, d.department_name FROM employees e LEFT JOIN departments d ON e.department_id = d.department_id WHERE e.salary > 50000;
```

SQL Best Practices

Use clear names and consistent formatting. Select only required columns. Use aliases for readability. Check joins carefully to avoid duplicate rows. Test UPDATE and DELETE with SELECT first. Handle NULL explicitly. Use transactions for related changes.